

QAOA

Quantum Approximate Optimization Algorithm

2026.06.10

2025113574 권나현

System Software Lab.

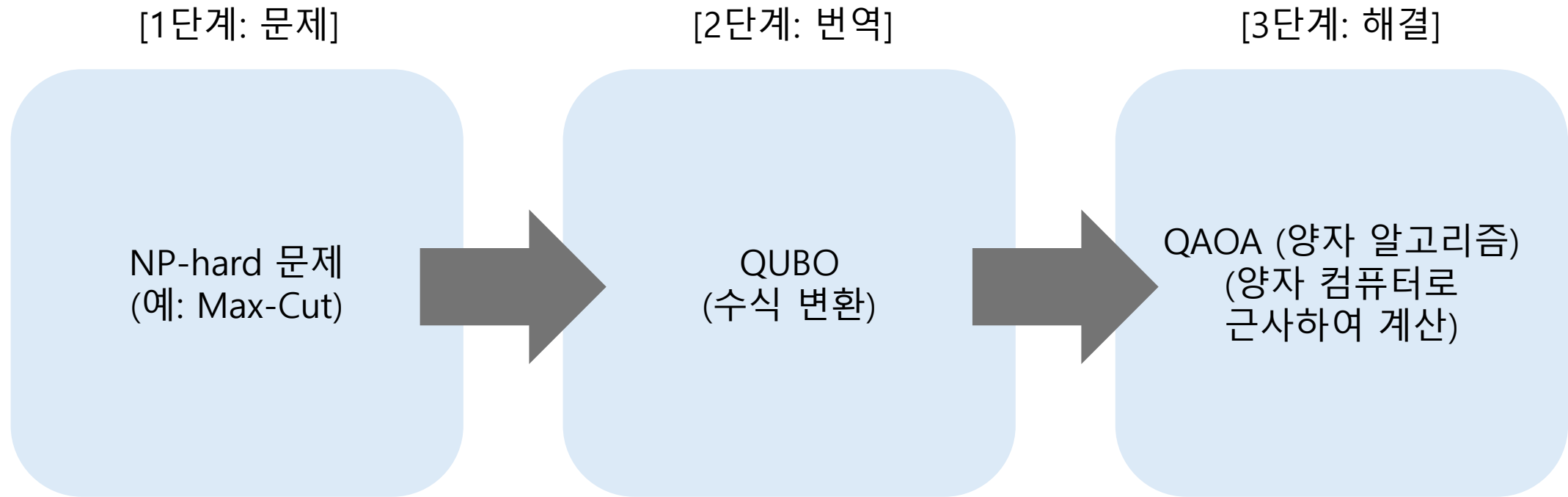
목차

1. 주제 선정 이유
2. 학습 배경 – 양자 최적화의 흐름
3. 문제 정의: max-cut(NP-hard)
4. QUBO란?
5. QAOA란?
6. Qiskit – max-cut 문제에 대한 5-node 실습

1. 주제 선정 이유

- 지난 시간에 사용법을 익혔던 QPU를 실제 알고리즘 문제를 푸는 데 직접 적용해 보고자 하였다.
- QAOA는 다항 시간 안에 근사해를 높은 확률로 찾아내어 계산 속도를 획기적으로 단축시킨다는 장점이 있다.

2. 학습 배경 – 양자 최적화의 흐름

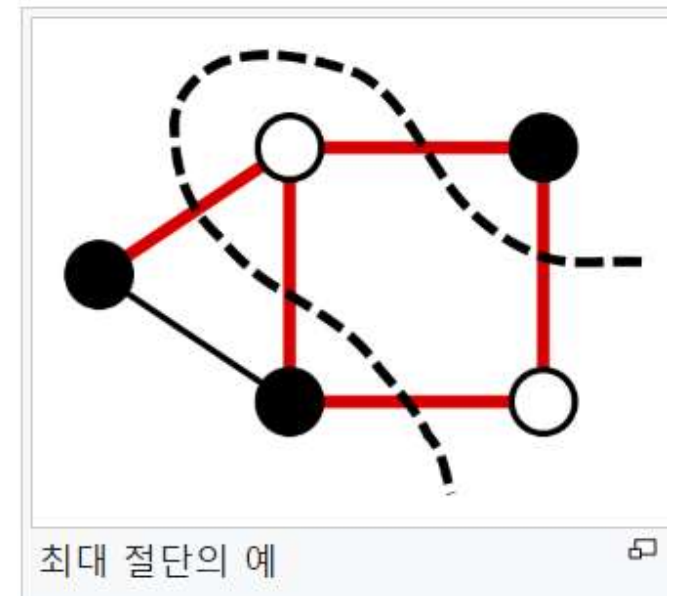
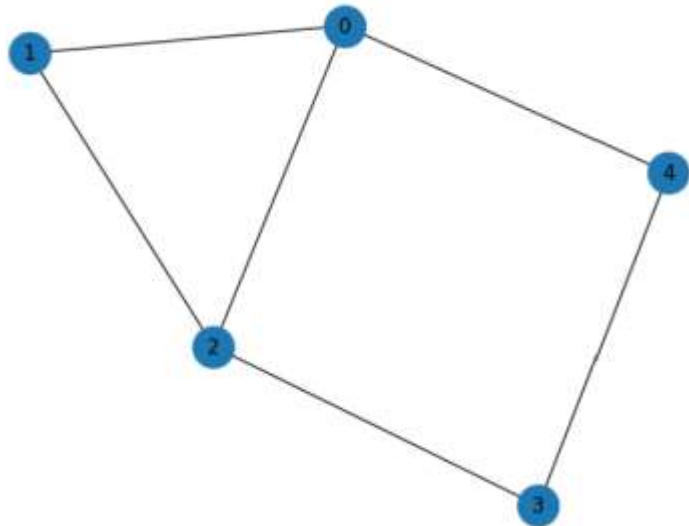


3. 문제 정의: max-cut(NP-hard)

- NP-hard (NP-난해): 컴퓨터 과학에서 모든 NP(비결정적 다항시간) 문제만큼 어렵거나 그보다 더 어려운 문제들의 집합
- Max-cut problem (최대 절단 문제)

그래프의 정점을 두 집합으로 분할하면서
두 집합 사이의 간선 수를 최대한 크게 만드는 절단을 찾는 문제

Ex)



4. QUBO란?

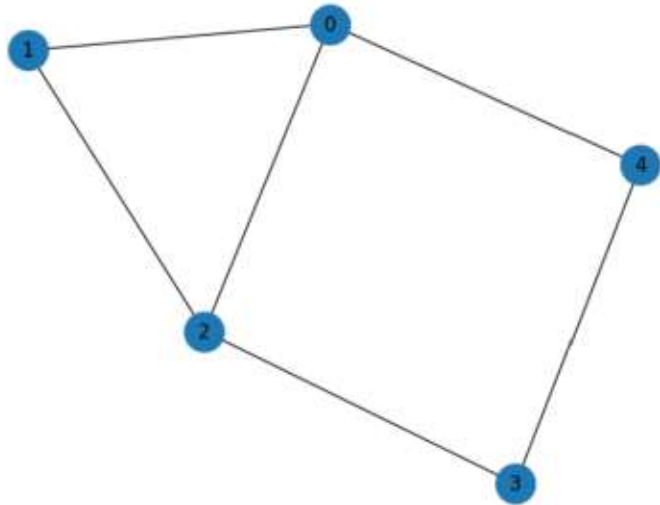
- Quadratic Unconstrained Binary Optimization (이차 무제약 이진 최적화 문제)
- 세부적 의미
 - Quadratic(이차식): 변수 간의 상호작용이 존재하는 비용 함수
 - Unconstrained(무제약): 제약 조건을 패널티 항으로 식 내부에 포함하여 제약 조건을 제거
 - Binary(이진 변수): 미지수 $\rightarrow$ 의사결정(선택 여부)을 표현
 - Optimization (최적화 문제): 제한된 조건 속에서 목적 함수의 결과값을 최대화하거나 최소화하는 최적의 변수 조합을 찾는 과정
- 실체: 미지수가 0 또는 1인 이차식 = "비용 함수"
- 구성: 비용 식 + 패널티항

Ex) 제약조건: x_1, x_2 둘 중 하나는 반드시 1이어야 함 $\rightarrow x_1 + x_2 = 1$

패널티항 = $1000 \times (x_1 + x_2 - 1)^2$

최종식 = 원래 비용 식 + $1000 \times (x_1 + x_2 - 1)^2$

4.5. max-cut 문제의 QUBO



Edges

(0, 1)

(0, 2)

(0, 4)

(1, 2)

(2, 3)

(3, 4)

두 노드가 서로 다르다면 -> 1점(cut됨, 성공)

두 노드가 서로 같으면 -> 0점(cut 안됨, 실패)

$$\max_{x \in \{0,1\}^n} \sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j)$$

XOR 연산

$$\begin{aligned} &= (x_0 + x_1 - 2x_0 x_1) + (x_0 + x_2 - 2x_0 x_2) \\ &\quad + (x_0 + x_4 - 2x_0 x_4) + (x_1 + x_2 - 2x_1 x_2) \\ &\quad + (x_2 + x_3 - 2x_2 x_3) + (x_3 + x_4 - 2x_3 x_4) \end{aligned}$$

5. QAOA란?

- Quantum Approximate Optimization Algorithm (양자 근사 최적화 알고리즘)
- QUBO 문제를 양자컴퓨터를 적용한 고전-양자 하이브리드 알고리즘
- 파라미터: 베타, 감마
- 원리

QUBO의 수학적 관점: 비용 함수(Cost Function)의 값을 최소화하는 변수 찾기

QUBO의 양자물리학적 관점: 해밀토니안(H)의 에너지 상태를 최소화하는 바닥상태 찾기

비용 해밀토니안: 구하고 싶은 비용 함수(QUBO 등)을

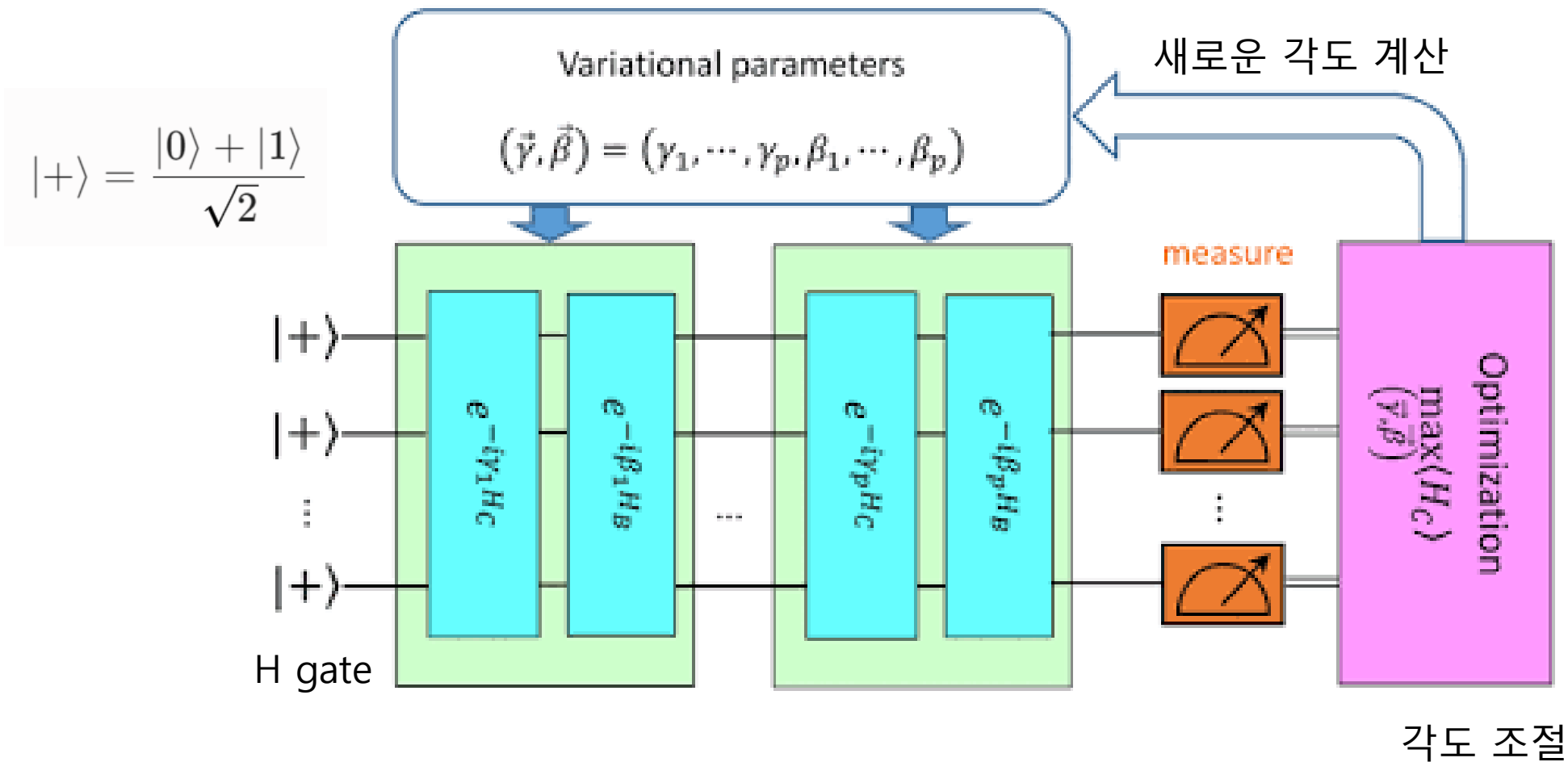
양자계의 에너지 식(해밀토니안)으로 그대로 모방해서 설계한 것

비용이 가장 낮은 최적의 답을 에너지가 가장 낮은 바닥 상태와 일치하도록 양자 시스템을 설계

→ 양자 컴퓨터가 비용 해밀토니안의 바닥 상태를 찾아내면 최적화 문제의 정답을 얻는 원리

5. QAOA란?

그림으로 보는 QAOA의 원리



5. QAOA란?

복소수 지수 형태: 회전 변환 행렬을 의미
(어떤 축을 기준으로 상태를 얼마만큼 회전)

QAOA 루프의 동작 과정

1. 초기 중첩 상태 $H^{\otimes n}|0\rangle$: 모든 큐비트에 H 게이트 적용
2. 교대 층(alternating layers): 2개의 레이어를 번갈아가며 반복적으로 적용

Qiskit
Runtime sessions

cost operator:

$$e^{-i\gamma_p H_c}$$

시간(각도) 매개변수
비용 해밀토니안

mixer operator:

$$e^{-i\beta_p H_m}$$

간섭 유발 (상태 섞음)

회전 각도 파라미터

$$\gamma_p, \beta_p$$

는 고전적인 피드백 루프에서 최적화됨

(양자컴퓨터에서는 큐비트를 마이크로펄스로 회전시킨다)

5. QAOA란?

- QAOA의 원리 - 비용 해밀토니안

해밀토니안 H: System의 전체 에너지를 나타내는 수학적 연산자

바닥 상태(Ground State): 에너지가 가장 낮은 안정적인 상태

어떤 양자계의 **해밀토니안을 계산** → 그 계가 가질 수 있는 **에너지 상태와 바닥 상태를 찾음**

실체: 거대한 대각행렬

$$H_C = \sum_{ij} Q_{ij} Z_i Z_j + \sum_i b_i Z_i$$

Q_{ij} : 상호작용 가중치 (Interaction Term)

b_i : 단독 가중치 (Linear Term)

복습: pauli Z gate

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

바닥 상태?

$$Z_1 Z_2 = Z \otimes Z = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

매핑

$|00\rangle$

$|01\rangle$

$|10\rangle$

$|11\rangle$



5.5. max-cut 문제의 QAOA

고전 컴퓨터용 QUBO 수식을 양자 컴퓨터용 비용 해밀토니안 으로 변환

$$\max_{x \in \{0,1\}^n} \sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j) \quad \longrightarrow \quad H_C = \sum_{ij} Q_{ij} Z_i Z_j + \sum_i b_i Z_i$$

$$x_i = \frac{1 - Z_i}{2} \quad \text{치환}$$

$$Z|0\rangle = +1$$

$$Z|1\rangle = -1 \quad \rightarrow \text{간선을 } Z \text{로 표현하는 이유}$$

$$(Z = +1) : x_i = \frac{1-1}{2} = 0$$

$$(Z = -1) : x_i = \frac{1-(-1)}{2} = 1$$

$$\begin{aligned} x_i + x_j - 2x_i x_j &= \left(\frac{1-Z_i}{2}\right) + \left(\frac{1-Z_j}{2}\right) - 2\left(\frac{1-Z_i}{2}\right)\left(\frac{1-Z_j}{2}\right) \\ &= \frac{1-Z_i}{2} + \frac{1-Z_j}{2} - \frac{(1-Z_i)(1-Z_j)}{2} \\ &= \frac{1-Z_i + 1-Z_j - (1-Z_j - Z_i + Z_i Z_j)}{2} \\ &= \frac{1 - Z_i Z_j}{2} = \frac{1}{2}I - \frac{1}{2}Z_i Z_j \end{aligned}$$

$$H_C = \sum_{(i,j)} \left(\frac{1}{2}I - \frac{1}{2}Z_i Z_j\right) \quad (Q_{ij} = -\frac{1}{2}, b_i = 0)$$

6. Qiskit: Max-cut 5 node 실습

- 필요한 라이브러리

1. rx, draw_graph -> 최대 컷 문제의 그래프 정의
2. SprasePauliOp -> 그래프를 수학적 수식(해밀토니안)으로 변환
3. QAOAAnstaz -> 수식을 해결하기 위한 양자 회로 생성
4. pass_manager -> 실제 IBM 양자 하드웨어 맞춤형으로 회로 변환(트랜스파일)
5. Session + EstimatorV2 -> (반복) EstimatorV2: 기댓값 측정, minimize(SciPy)로 고전 컴퓨터가 파라미터 업데이트
6. SamplerV2 -> 최적의 파라미터로 최종 정답(비트스트림) 샘플링 및 확인

6. Qiskit: Max-cut 5 node 실습

- Max-cut 문제 그래프 그리기

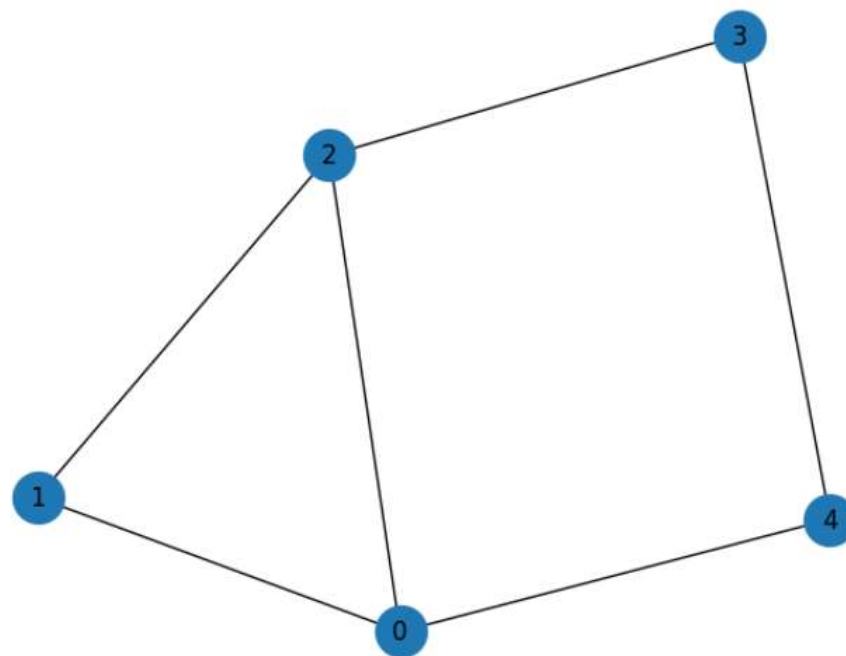
```
n_small = 5

graph = rx.PyGraph()
graph.add_nodes_from(np.arange(0, n_small, 1)) # 노드 그리기

# 간선 정보 리스트
edge_list = [
    (0, 1, 1.0),
    (0, 2, 1.0),
    (0, 4, 1.0),
    (1, 2, 1.0),
    (2, 3, 1.0),
    (3, 4, 1.0),
]

graph.add_edges_from(edge_list) # 간선 그리기

draw_graph(graph, node_size=600, with_labels=True) # 그래프 그리기
```



6. Qiskit: Max-cut 5 node 실습

- Max-cut 문제를 비용 해밀토니안으로 변환

```
def build_max_cut_paulis(graph):  
    pauli_list = []  
    for edge in list(graph.edge_list()):  
        weight = graph.get_edge_data(edge[0], edge[1])  
        pauli_list.append(("ZZ", [edge[0], edge[1]], weight))  
    return pauli_list
```

튜플들로 이루어진 리스트 형태

```
max_cut_paulis = build_max_cut_paulis(graph)  
cost_hamiltonian = SparsePauliOp.from_sparse_list(max_cut_paulis, n_small)  
print("Cost Function Hamiltonian: ", cost_hamiltonian)
```

→ `#.edge_list()`: 가중치를 제외한 노드 연결 정보를 반환하는 rustworkx의 내장 함수
`print(graph.edge_list())`

• `EdgeList[(0, 1), (0, 2), (0, 4), (1, 2), (2, 3), (3, 4)]`

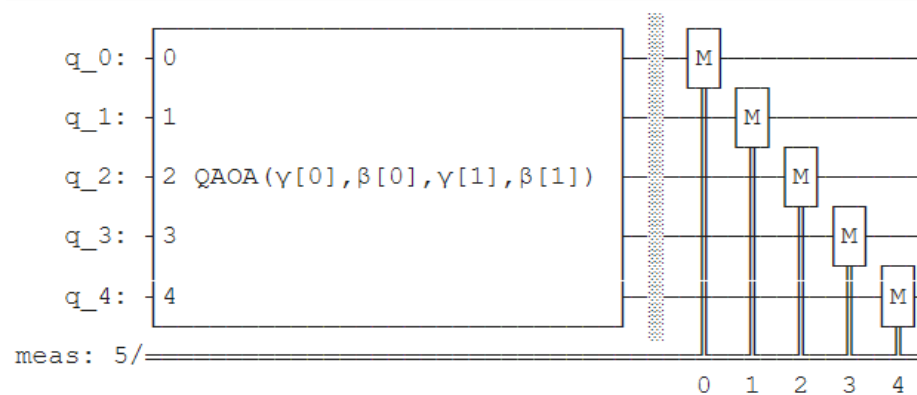
... `[('ZZ', [0, 1], 1.0), ('ZZ', [0, 2], 1.0), ('ZZ', [0, 4], 1.0), ('ZZ', [1, 2], 1.0), ('ZZ', [2, 3], 1.0), ('ZZ', [3, 4], 1.0)]`
Cost Function Hamiltonian: `SparsePauliOp([('IIIZZ', 'IIZIZ', 'ZIIIZ', 'IIZZI', 'IZZII', 'ZZIII'),`
`coeffs=[1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j])` coefficient 가중치

6. Qiskit: Max-cut 5 node 실습

- 도출한 cost hamiltonian을 바탕으로 QAOA ansatz(초기 구조) 회로 생성하기

```
circuit = QAOAAnsatz(cost_operator=cost_hamiltonian, reps=2)
circuit.measure_all()
circuit.draw()
```

QAOAAnsatz(): 비용 해밀토니안의 구조를 양자 게이트 회로로 번역하는 함수
실체: 앞에서 봤던 파울리 z 행렬연산을 대신 수행함



```
circuit.parameters
```

```
ParameterView([ParameterVectorElement( $\beta[0]$ ), ParameterVectorElement( $\beta[1]$ ), ParameterVectorElement( $\gamma[0]$ ), ParameterVectorElement( $\gamma[1]$ )])
```


6. Qiskit: Max-cut 5 node 실습

- 양자 하드웨어 실행을 위한 문제 최적화

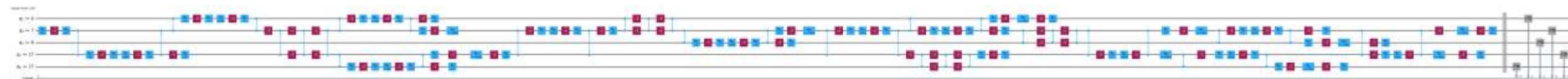
```
# IBM 계정 연결(최초 1회)
QiskitRuntimeService.save_account(
    channel="ibm_quantum_platform",
    token="토큰",
    instance="crn"
)

service = QiskitRuntimeService()
backend = service.least_busy(operational=True, simulator=False, min_num_qubits=127)
print(backend)

pm = generate_preset_pass_manager(optimization_level=3, backend=backend)

candidate_circuit = pm.run(circuit)
candidate_circuit.draw("mpl", fold=False, idle_wires=False)
```

트랜스파일 완료된 최종 양자회로



6. Qiskit: Max-cut 5 node 실습

- 현재 양자 상태의 점수를 계산하는 함수 (Optimization 단계)

`initial_gamma = np.pi` 초기 설정: 정답인 조합들의 위상(Phase)만 완벽하게 180도 돌려놓아
`initial_beta = np.pi / 2` 오답과 기하학적으로 가장 멀어지게 설정하는 각도. 이 상태에서 믹서 레이어가 섞어주면 간섭이 극대화
`init_params = [initial_beta, initial_beta, initial_gamma, initial_gamma]`

```
...  
비용 함수: 매개변수 각도(params)에서의 에너지 기댓값(cost) 반환  
params: 테스트할 회전 게이트들의 각도 리스트  
ansatz: 각도 변수들을 포함하는 양자회로 템플릿  
hamiltonian: 비용 함수 연산자  
estimator: 기댓값을 계산할 프리미티브 객체  
...  
def cost_func_estimator(params, ansatz, hamiltonian, estimator):  
    isa_hamiltonian = hamiltonian.apply_layout(ansatz.layout)  
  
    pub = (ansatz, isa_hamiltonian, params)  
    job = estimator.run([pub])  
  
    results = job.result()[0]  
    cost = results.data.evs #기댓값(Expectation Values) = 비용  
    objective_func_vals.append(cost)  
  
    return cost
```

QPU 측정을 호출

실제 루프에서 이 함수를 여러 번 호출해서 반복적으로 실행한다.

6. Qiskit: Max-cut 5 node 실습

- 실제 루프 설계

```
objective_func_vals = []
```

```
estimator = EstimatorV2(mode=backend)
estimator.options.default_shots = 1000
estimator.options.dynamical_decoupling.enable = True
estimator.options.dynamical_decoupling.sequence_type = "XY4"
estimator.options.twirling.enable_gates = True
estimator.options.twirling.num_randomizations = "auto"
estimator.options.environment.job_tags = ["TUT_QAOA"]
```

기댓값 도출을 위한 내부 루프

#scipy.optimize.minimize 함수 사용: 고전 컴퓨터와 양자 컴퓨터 사이의 루프 제어

```
result = minimize(
    cost_func_estimator, # 비용함수 측정 함수
    init_params, # 초기 각도
    args=(candidate_circuit, cost_hamiltonian, estimator),
    method="COBYLA", # 각도를 수정할 수학적 알고리즘
    tol=1e-2, # 오차자 0.01 이하로 줄어들면 루프 종료
)
```

```
print(result)
```

```
message: Return from COBYLA because the trust region radius reaches its lower bound.
success: True
status: 0
fun: -2.2277201429177835
x: [ 1.918e+00  1.364e+00  2.770e+00  4.086e+00]
nfev: 30
maxcv: 0.0
```

각도
함수 호출 횟수(외부 루프)

☐	ID	Status	Instance	Mode	Created	↓	Completed	QPU	Usage	User	Tags
☐	d8aou0Bb1oec7...	Completed	ghz-test	Job	5/26/26, 9:16 PM		5/26/26, 9:16 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotres3bbc7...	Completed	ghz-test	Job	5/26/26, 9:15 PM		5/26/26, 9:15 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotlqvmpc7...	Completed	ghz-test	Job	5/26/26, 9:15 PM		5/26/26, 9:15 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotd2a5s8s7...	Completed	ghz-test	Job	5/26/26, 9:14 PM		5/26/26, 9:15 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aot82vmpc7...	Completed	ghz-test	Job	5/26/26, 9:14 PM		5/26/26, 9:14 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aot2ob1oec7...	Completed	ghz-test	Job	5/26/26, 9:14 PM		5/26/26, 9:14 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotms3bbc7...	Completed	ghz-test	Job	5/26/26, 9:13 PM		5/26/26, 9:14 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotms3bbc...	Completed	ghz-test	Job	5/26/26, 9:13 PM		5/26/26, 9:13 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotjqas8s7...	Completed	ghz-test	Job	5/26/26, 9:13 PM						
☐	d8aotms3bbc7...	Completed	ghz-test	Job	5/26/26, 9:12 PM						

Items per page: 10

48 items

☐	ID	Status	Instance	Mode	Created	↓	Completed	QPU	Usage	User	Tags
☐	d8aot9ob1oec7...	Completed	ghz-test	Job	5/26/26, 9:12 PM		5/26/26, 9:12 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aot4ia5s8s7...	Completed	ghz-test	Job	5/26/26, 9:12 PM		5/26/26, 9:12 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotwvmpc7...	Completed	ghz-test	Job	5/26/26, 9:11 PM		5/26/26, 9:12 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotmqas8s7...	Completed	ghz-test	Job	5/26/26, 9:11 PM		5/26/26, 9:11 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotqas8s7...	Completed	ghz-test	Job	5/26/26, 9:10 PM		5/26/26, 9:11 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotms3bbc...	Completed	ghz-test	Job	5/26/26, 9:02 PM		5/26/26, 9:10 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotms3bbc7...	Completed	ghz-test	Job	5/26/26, 9:02 PM		5/26/26, 9:02 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotms3bbc7...	Completed	ghz-test	Job	5/26/26, 9:01 PM		5/26/26, 9:02 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotms3bbc7...	Completed	ghz-test	Job	5/26/26, 9:01 PM		5/26/26, 9:01 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotms3bbc7...	Completed	ghz-test	Job	5/26/26, 8:54 PM		5/26/26, 8:54 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA

5 pages

☐	ID	Status	Instance	Mode	Created	↓	Completed	QPU	Usage	User	Tags
☐	d8aot70b1oec7...	Completed	ghz-test	Job	5/26/26, 8:53 PM		5/26/26, 8:53 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aot1aa5s8s7...	Completed	ghz-test	Job	5/26/26, 8:52 PM		5/26/26, 8:52 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotaa5s8s73...	Completed	ghz-test	Job	5/26/26, 8:52 PM		5/26/26, 8:52 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotles3bbc73...	Completed	ghz-test	Job	5/26/26, 8:51 PM		5/26/26, 8:52 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotges3bbc7...	Completed	ghz-test	Job	5/26/26, 8:51 PM		5/26/26, 8:51 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotb2vmpc7...	Completed	ghz-test	Job	5/26/26, 8:51 PM		5/26/26, 8:51 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aot52a5s8s7...	Completed	ghz-test	Job	5/26/26, 8:50 PM		5/26/26, 8:51 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotqvmpc...	Completed	ghz-test	Job	5/26/26, 8:50 PM		5/26/26, 8:50 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotpaas8s7...	Completed	ghz-test	Job	5/26/26, 8:49 PM		5/26/26, 8:50 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA
☐	d8aotjqas8s7...	Completed	ghz-test	Job	5/26/26, 8:49 PM		5/26/26, 8:49 PM	ibm_kingston	11s	Nahyeon Kwon	TUT_QADA

Items per page: 10

48 items

5 pages

6. Qiskit: Max-cut 5 node 실습

- 도출한 각도를 사용하여 확인

▶ `optimal_params = [1.918, 1.364, 2.770, 4.086]`
`optimized_circuit = candidate_circuit.assign_parameters(optimal_params)`
`optimized_circuit.draw("mpl", fold=False, idle_wires=False)`



최종 최적화된 매개변수(Optimal Parameters)들을 실제 양자 회로에 바인딩(Binding)한 회로

6. Qiskit: Max-cut 5 node 실습

- 최적의 각도를 적용한 최종 양자 회로를 실행하여 정답(최적해) 확인

```
sampler = Sampler(mode=backend)
sampler.options.default_shots = 10000 # 회로 루프의 반복 횟수

# 오류 제어 옵션 설정
sampler.options.dynamical_decoupling.enable = True # 동적 디커플링
sampler.options.dynamical_decoupling.sequence_type = "XY4"
sampler.options.twirling.enable_gates = True # 파울리 트윙링
sampler.options.twirling.num_randomizations = "auto"

sampler.options.environment.job_tags = ["TUT_QAOA"] # 작업 태그 지

pub = (optimized_circuit,) # 원소 1개짜리 튜플 생성
job = sampler.run([pub], shots=int(1e4)) #pub을 양자 컴퓨터에서 10000번 실행
counts_int = job.result()[0].data.meas.get_int_counts()
counts_bin = job.result()[0].data.meas.get_counts()
shots = sum(counts_int.values())
final_distribution_int = {key: val / shots for key, val in counts_int.items()}
final_distribution_bin = {key: val / shots for key, val in counts_bin.items()}
print(final_distribution_int)
```

... {26: 0.0606, 22: 0.1195, 24: 0.0084, 11: 0.1173, 9: 0.1104, 19: 0.0257, 5: 0.0637, 10: 0.0352, 8: 0.0088, 21: 0.0321, 14: 0.0235, 29: 0.0129, 17: 0.021, 13: 0.0304, 20: 0.1036, 12: 0.02

확률분포가 출력됨

6. Qiskit: Max-cut 5 node 실습

- 이전의 확률분포 데이터에서 가장 확률이 높은 비트열을 가공, 출력하기

```
def to_bitstring(integer, num_bits):  
    result = np.binary_repr(integer, width=num_bits) # 10진수를 2진수 비트열로 변환  
    return [int(digit) for digit in result]
```

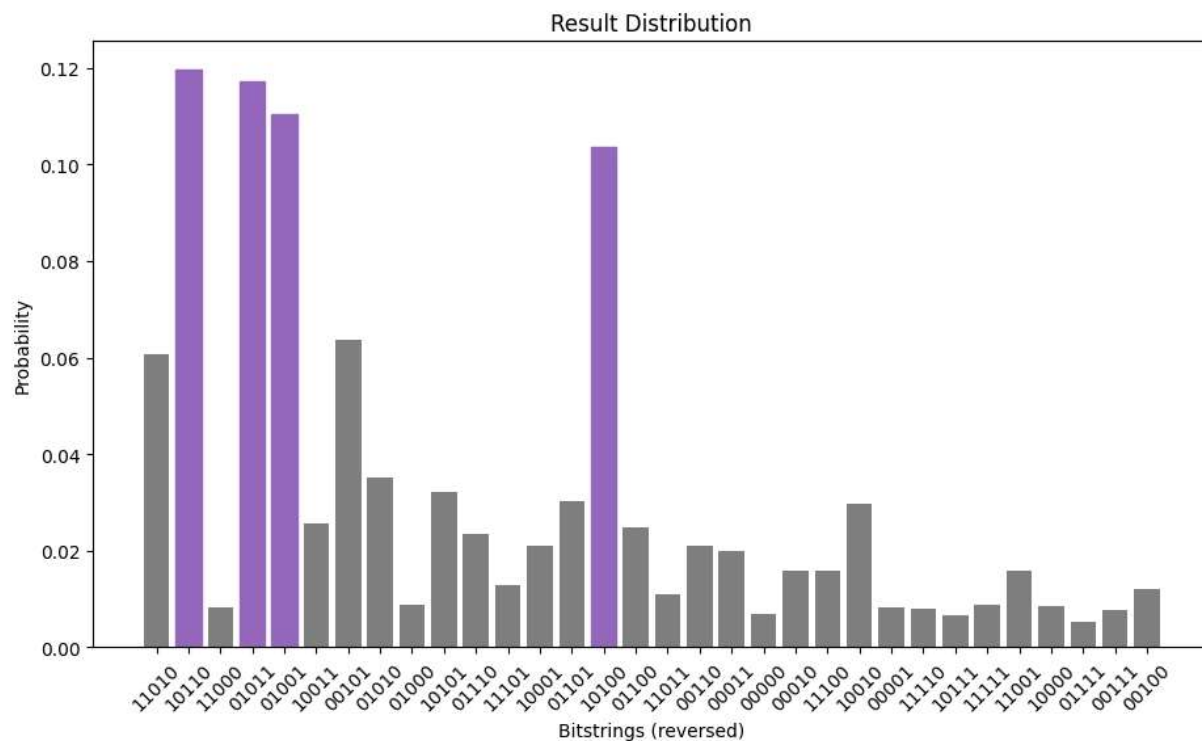
```
keys = list(final_distribution_int.keys())  
values = list(final_distribution_int.values())  
most_likely = keys[np.argmax(np.abs(values))] # 가장 높은 확률의 데이터 추출  
most_likely_bitstring = to_bitstring(most_likely, len(graph))  
most_likely_bitstring.reverse() # 리틀 엔디안 되돌리기  
  
print("Result bitstring:", most_likely_bitstring)
```

```
... Result bitstring: [0, 1, 1, 0, 1]
```

6. Qiskit: Max-cut 5 node 실습

- 결과값을 그래프로 출력하기

```
plt.rcParams.update({"font.size": 10})
final_bits = final_distribution_bin
values = np.abs(list(final_bits.values()))
top_4_values = sorted(values, reverse=True)[:4]
positions = []
for value in top_4_values:
    positions.append(np.where(values == value)[0])
fig = plt.figure(figsize=(11, 6))
ax = fig.add_subplot(1, 1, 1)
plt.xticks(rotation=45)
plt.title("Result Distribution")
plt.xlabel("Bitstrings (reversed)")
plt.ylabel("Probability")
ax.bar(list(final_bits.keys()), list(final_bits.values()), color="tab:grey")
for p in positions:
    ax.get_children()[int(p[0])].set_color("tab:purple")
plt.show()
```

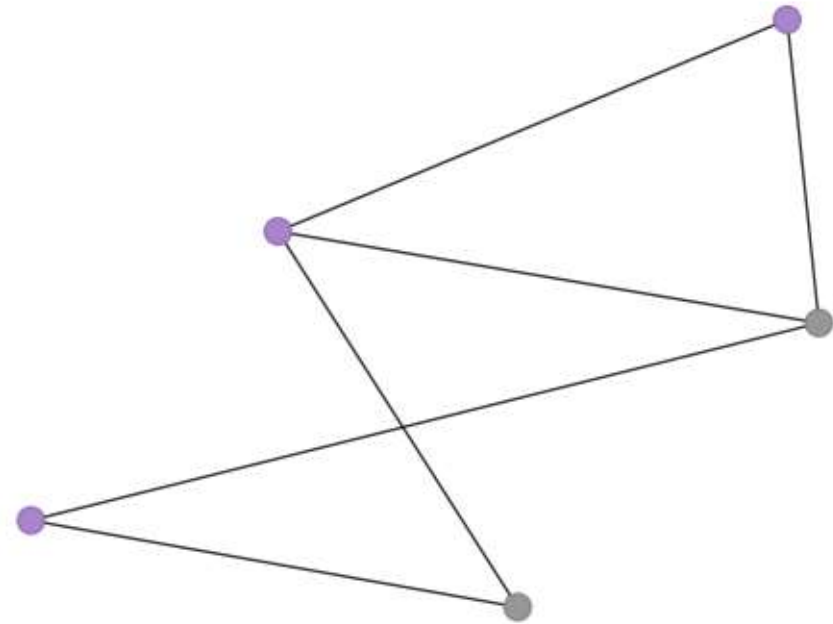


6. Qiskit: Max-cut 5 node 실습

- 결과값을 그래프로 출력하기

```
# auxiliary function to plot graphs
def plot_result(G, x):
    colors = ["tab:grey" if i == 0 else "tab:purple" for i in x]
    pos, _default_axes = rx.spring_layout(G), plt.axes(frameon=True)
    rx.visualization.mpl_draw(
        G, node_color=colors, node_size=100, alpha=0.8, pos=pos
    )
```

```
plot_result(graph, most_likely_bitstring)
```



감사합니다.